

Q1.2

Solution/Output:

The solution of Q1.1.a (DFS) is:

Solution Path: [(3, 3, 0, 0, 'L'), (2, 2, 1, 1, 'R'), (3, 2, 0, 1, 'L'), (3, 0, 0, 3, 'R'), (3, 1, 0, 2, 'L'), (1, 1, 2, 2, 'R'), (2, 2, 1, 1, 'L'), (0, 2, 3, 1, 'R'), (0, 3, 3, 0, 'L'), (0, 1, 3, 2, 'R'), (0, 2, 3, 1, 'L'), (0, 0, 3, 3, 'R')]

Total cost = 11

Number of node expansions = 13

The solution of Q1.1.b (BFS) is:

Solution Path: [(3, 3, 0, 0, 'L'), (3, 1, 0, 2, 'R'), (3, 2, 0, 1, 'L'), (3, 0, 0, 3, 'R'), (3, 1, 0, 2, 'L'), (1, 1, 2, 2, 'R'), (2, 2, 1, 1, 'L'), (0, 2, 3, 1, 'R'), (0, 3, 3, 0, 'L'), (0, 1, 3, 2, 'R'), (1, 1, 2, 2, 'L'), (0, 0, 3, 3, 'R')]

Total cost = 11

Number of node expansions = 14

For this problem there were some notable similarities between DFS and BFS. They both found a solution with a total cost of 11. However, they explored the state spaces differently. DFS follows one branch directly, the whole way through until there are no more nodes to expand. BFS all states on one depth before expanding to the next level. Since every move in this version of the game has a cost of 1 BFS is guaranteed finding the solution at the shallowest depth, the fewest moves, which for this version is optimal. DFS happened to find a solution with cost of 11 but it is not guaranteed it would find this optimal solution since it just stays on one path. It could have found a longer nonoptimal solution.

The node expansion count was also very similar with DFS at 13 and BFS at 14. This means that DFS was slightly more efficient for this search. But, this does not guarantee that DFS will always be more efficient. DFS finds the first solution and that may be a deeper depth than the most optimal one and that is what determines its efficiency. If the solution was at a shallow depth then BFS would likely be more efficient. BFS more expansions here shows the tradeoff with guaranteeing a optimal solution for possible more cost.

Q2.2

The solution of Q2.1 (UCS, cost model A) is:

Solution Path: [(3, 3, 0, 0, 'L'), (3, 1, 0, 2, 'R'), (3, 2, 0, 1, 'L'), (3, 0, 0, 3, 'R'), (3, 1, 0, 2, 'L'), (1, 1, 2, 2, 'R'), (2, 2, 1, 1, 'L'), (0, 2, 3, 1, 'R'), (0, 3, 3, 0, 'L'), (0, 1, 3, 2, 'R'), (0, 2, 3, 1, 'L'), (0, 0, 3, 3, 'R')]

Total cost = 23

Number of node expansions = 14

The solution of Q2.1 (UCS, cost model B) is:

Solution Path: [(3, 3, 0, 0, 'L'), (3, 1, 0, 2, 'R'), (3, 2, 0, 1, 'L'), (3, 0, 0, 3, 'R'), (3, 1, 0, 2, 'L'), (1, 1, 2, 2, 'R'), (2, 2, 1, 1, 'L'), (0, 2, 3, 1, 'R'), (0, 3, 3, 0, 'L'), (0, 1, 3, 2, 'R'), (0, 2, 3, 1, 'L'), (0, 0, 3, 3, 'R')]

Total cost = 17

Number of node expansions = 14

Compared to BFS, UCS will probably choose a different sequence of actions because it looks at the cost of actions instead of just the number of moves. In this case the relationship between path and cost is shown by both of them finding a solution with 11 steps, but the paths are different since they evaluate cost differently. Cost model A had a cost of 23 and Cost model B had a cost of 17 but all 3 (A, B, and BFS) all expanded 14 nodes.

UCS is important when actions have different costs. BFS is guaranteed to work when all moves have the same cost because the shortest path will be the most efficient. When certain moves cost more this isn't always true. The main tradeoff is UCS can sometimes expand more nodes than BFS because it may explore several low-cost paths before reaching the goal.

Q3.2 The Writing Component Part A — Admissibility Proofs

Heuristic 1 is admissible because it never overestimates the real cost of move the remaining cannibals and missionaries. $h1 = 2 * M_l + C_l$ with M_l and C_l being the missionaries and cannibals left. In this model every missionary must cross the river costing 2 and every cannibal must cross costing 1. This allows it to underestimate because it does not account for any of the return trips required to move the boat, and since the missionaries can not be out numbered there are more return trips required than just to move everyone. Since it always underestimates it is admissible.

Heuristic 2 is admissible because it never overestimates the real cost of move the remaining cannibals and missionaries. $h2 = \text{ceil}[(2 * M_l + C_l) / 3]$. Since we know the inner term of this equation, $2 * M_l + C_l$, is already less than the real remaining cost then this one must be as well because it is divided by 3. Essentially meaning that crossing each missionary for a cost of at most 1 and cannibal for 1 since we are rounding up. This is less than the actual because it does not account for trips back. Since it always under estimates it is admissible.

Part B —

The solution of Q3.1 (Heuristic 1) is:

Solution Path: [(3, 3, 0, 0, 'L'), (3, 1, 0, 2, 'R'), (3, 2, 0, 1, 'L'), (3, 0, 0, 3, 'R'), (3, 1, 0, 2, 'L'), (1, 1, 2, 2, 'R'), (2, 2, 1, 1, 'L'), (0, 2, 3, 1, 'R'), (0, 3, 3, 0, 'L'), (0, 1, 3, 2, 'R'), (0, 2, 3, 1, 'L'), (0, 0, 3, 3, 'R')]

Total cost = 23

Number of node expansions = 13

The solution of Q3.1 (Heuristic 2) is:

Solution Path: [(3, 3, 0, 0, 'L'), (3, 1, 0, 2, 'R'), (3, 2, 0, 1, 'L'), (3, 0, 0, 3, 'R'), (3, 1, 0, 2, 'L'), (1, 1, 2, 2, 'R'), (2, 2, 1, 1, 'L'), (0, 2, 3, 1, 'R'), (0, 3, 3, 0, 'L'), (0, 1, 3, 2, 'R'), (0, 2, 3, 1, 'L'), (0, 0, 3, 3, 'R')]

Total cost = 23

Number of node expansions = 14

Heuristic	Solution Path	Total Cost	Node Expansions	Same cost as UCS Cost model A
H1	See above	23	13	Yes
H2	See above	23	14	yes
H3	See below	23	13	yes

d) h1 was slightly more efficient. It expanded 13 node while h2 expanded 14. This is because h1 gives a stronger estimate of the remaining cost. As said above, h2 is always less than h1 and since they are both admissible than h1 is better.

Part C — Designing a Tighter Heuristic

The solution of Q3.1 (Heuristic 3) is:

Solution Path: [(3, 3, 0, 0, 'L'), (3, 1, 0, 2, 'R'), (3, 2, 0, 1, 'L'), (3, 0, 0, 3, 'R'), (3, 1, 0, 2, 'L'), (1, 1, 2, 2, 'R'), (2, 2, 1, 1, 'L'), (0, 2, 3, 1, 'R'), (0, 3, 3, 0, 'L'), (0, 1, 3, 2, 'R'), (0, 2, 3, 1, 'L'), (0, 0, 3, 3, 'R')]

Total cost = 23

Number of node expansions = 13

$$H3 = 2M_L + C_L + 2 \max(0, (M_L + C_L) - 2 \text{ [if boat == "L"]})$$

.The - 2 is only when the boat is on the left to account for the possibility of 2 people traveling.

H3 builds on h1 by adding a term accounting for return trips. The new term is the cost if each remaining person on the left had to have someone come back to get them and then their move to the right.

This heuristic is admissible because it still only counts costs that must occur in any solution. It assumes every required return trip can use the cheapest possible passenger and ignores other restrictions, such as the missionaries and cannibals safety rules, and the extra cost of the missionaries which makes it a underestimate and thus admissible.

Since the new term is always ≥ 0 h3 will dominate h1 and thus will also dominate h2 because h1 does. In this case it found the same optimal cost of 23 while expanding 13 nodes.